

基于变系数热湿耦合模型的药材烘干时长确定

摘要

关键词：干基含水率；热湿耦合；材料坐标；有限体积法；烘干终点判据

热风烘干中药材内部温湿场具有显著的时空非均匀性，烘干终点还受相变潜热、收缩几何与边界条件影响。针对题目给定的烘房温湿度历程、药材半径历程和经验公式及物性，本文以干基含水率为状态变量，以药材干物质质量守恒为关键约束，建立圆柱坐标下受相变潜热影响的变系数热湿耦合模型；对收缩问题引入材料坐标，采用表面加密有限体积离散和隐式 BDF 方法求解，并以全场最大干基含水率首次不高于 0.15 kg/kg 作为达标判据。针对问题一，按预热阶段物性计算 30 min 内的径向温度与含水率分布， 1800 s 时中心、表面温度分别为 $11.2204\text{ }^{\circ}\text{C}$ 和 $12.7079\text{ }^{\circ}\text{C}$ ，表面含水率为 1.5102 kg/kg 。针对问题二，采用全程热湿耦合物性从初始状态独立求解， 3 h 时中心、表面温度为 $31.1631\text{ }^{\circ}\text{C}$ 和 $31.7414\text{ }^{\circ}\text{C}$ ，中心、表面含水率分别为 2.2680 kg/kg 和 0.8882 kg/kg 。针对问题三，以各处均达标为终点，得到临界烘干时长为 60.430002 h ，严格达标的首个整分钟为 $60\text{ h }26\text{ min}$ 。针对问题四，将实测半径引入材料坐标模型，得到临界烘干时长为 56.501190 h ，严格达标的首个整分钟为 $56\text{ h }31\text{ min}$ 。网格加密与 BDF-Radau 交叉计算表明，末档网格时长变化不超过 0.045 s 、两积分器差异不超过 0.0014 s ；灵敏度分析表明扩散系数和后期温度是影响烘干时长的主要因素。模型结果适用于题给物性和有效边界闭合条件，不将数值收敛等同于实验预测精度。

一、 问题重述

1.1 问题背景

热风烘干是中药材干燥工序中的常见方式，分为预热平衡与恒温干燥两个阶段。工艺参数选取不当会造成干燥效率低、能耗高、成品品质不稳定，而传统的试验优化方法成本高、周期长，因此需要借助数理分析与数值仿真获得干燥规律，以降低能耗、提高效率并稳定品质。热风烘干设备的结构如图 1 所示。



图 1 中药材热风烘干机

1.2 问题提出

已知药材的尺寸、初始温度与初始含水率，以及烘房温湿度和药材半径随时间的变化数据，并给出各阶段的物性经验公式。需要解决的问题如下。

问题一：要求结合附录二的物性，建立预热平衡阶段药材温度与水分浓度变化的数学模型，给出一定时间内药材沿径向的温度、水分浓度数据。

问题二：在问题一的基础上，扩展至整个烘干阶段，结合附录三中的经验公式，给出一定时间内药材沿径向的温度与水分浓度模型。

问题三、四：均需要判定烘干的终点。问题三给出了药材烘干达标所需的水分浓度边界（上限），需求解烘干药材所需时长，并给出一定时间内药材沿径向的水分浓度变化数据。问题四考虑了前三问忽略的药材尺寸收缩的因素，需综合考虑温度、水分浓度和药材半径之间的相互关系。

四个问题在局部传递、全程耦合、终点判定与几何变化上依次递进，其关系如图 2 所示。

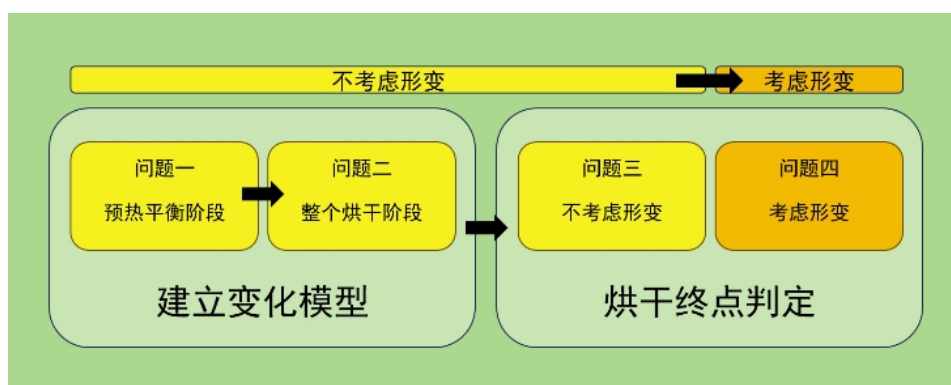


图 2 四个问题间的关系

二、 问题分析

2.1 问题一的分析

预热平衡阶段的扩散系数只依赖含水率，温度则通过导热与对流影响热场，热湿间的耦合较弱。求解的关键有三点：扩散系数随含水率呈指数衰减，属于变系数扩散，需在散度内保留该系数；圆柱轴对称要求在中心处施加零通量条件，而不能直接计算 $1/r$ ；药材表面同时受到对流换热与对流传质控制，应采用第三类边界条件。注意到 30min 内含水率的变化集中在表层，求解微分方程时网格需在表面附近加密。

2.2 问题二的分析

与问题一不同，问题二的扩散系数同时依赖温度与含水率，热湿场产生双向耦合：温度决定水分迁移速率，水分输运与蒸发又改变温度。问题二覆盖整个烘干过程，并要求采用附录三的物性，不应以第一问的终点为起点应从 $t = 0$ 用同一组物性重新求解。题面展示窗口（3h）短于实际求解范围（2—3 天），输出应由同一连续解采样得到。

2.3 问题三的分析

问题三要求在给定判据下确定烘干时长，实质是带终止条件的求解问题。题面要求药材"各处"的水分浓度均低于 0.15kg/kg，故终点应取全场最大含水率首次降至该阈值的时刻。由于表层先干燥、中心最湿，若按平均含水率判定，将在中心尚未达标时提前停机，因此必须逐点比较并以全场最大值作为判据；停机时刻还应给出满足严格小于阈值的整分钟。

2.4 问题四的分析

问题四引入尺寸变化，几何不再是已知常数。附件只给出外半径，无法唯一确定内部应变，故采用引入参数最少的均匀径向收缩假设，并将方程改写材料坐标 $x = r/R(t)$ ，使移动边界固定为 $x = 1$ 。实测半径经保形插值后使用，以避免拟合振荡破坏单调性；温湿场不反向预测收缩，收缩仅作为已知输入。

三、 符号说明

本文主要符号的含义与单位见表 1。需要特别区分的是：药材含水率 C 以干药材质量为分母，空气含湿量 W 以干空气质量为分母，二者不能仅因单位形式相同而直接相减；环境平衡含水率 C_{eq} 是折算到药材干基定义后的驱动值，空气相对湿度 RH 也不是 C_{eq} 。

表 1 符号说明

符号	含义	单位
C	药材干基含水率（水分质量与干药材质量之比）	$\text{kg}\cdot\text{kg}^{-1}$
C_0	初始干基含水率（2.55）	$\text{kg}\cdot\text{kg}^{-1}$
C_s	药材表面干基含水率	$\text{kg}\cdot\text{kg}^{-1}$
C_{eq}	折算到药材干基的环境平衡含水率	$\text{kg}\cdot\text{kg}^{-1}$
W	空气含湿量（水蒸气质量与干空气质量之比）	$\text{kg}\cdot\text{kg}^{-1}$
RH	空气相对湿度	—
$T、T_a$	药材温度、烘房空气温度	$^{\circ}\text{C}$
T_K	热力学温度（ $T+273.15$ ）	K
$R、R_0$	药材当前半径、初始半径	m

符号	含义	单位
L	药材长度 (0.25)	m
r、x	径向坐标、材料坐标 ($x = r/R$)	m、—
D	有效水分扩散系数	$\text{m}^2 \cdot \text{s}^{-1}$
k	导热系数	$\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$
c_s 、 c_w	干物质比热、水的比热	$\text{J} \cdot \text{kg}^{-1} \cdot \text{K}^{-1}$
c_p	药材比热	$\text{J} \cdot \text{kg}^{-1} \cdot \text{K}^{-1}$
h、 h_m	对流换热系数 (25)、对流传质系数 (8×10^{-7})	$\text{W} \cdot \text{m}^{-2} \cdot \text{K}^{-1}$ 、 $\text{m} \cdot \text{s}^{-1}$
ρ_{emp}	题给经验密度 (用于初始质量与对照)	$\text{kg} \cdot \text{m}^{-3}$
ρ_d 、 ρ_{wet}	当前体积中的干质量密度、湿体积密度	$\text{kg} \cdot \text{m}^{-3}$
J_w 、 j_w	水分质量通量、表面水分质量通量	$\text{kg} \cdot \text{m}^{-2} \cdot \text{s}^{-1}$
F_w	控制体面上的水质量流率	$\text{kg} \cdot \text{s}^{-1}$
m_d	控制体干物质质量 (求解期间恒定)	kg
H	控制体显热能量 (以 0°C 为参考)	J
L_v	水的汽化潜热 (2.38×10^6)	$\text{J} \cdot \text{kg}^{-1}$
λ	表面汽化耦合系数 ($0 \leq \lambda \leq 1$)	—
a_w 、 β_p	水活度、界面传质系数 (本文未标定)	—、 $\text{kg} \cdot \text{m}^{-2} \cdot \text{s}^{-1} \cdot \text{Pa}^{-1}$
N、 w_i 、 V_i	径向控制体数、控制体权重、控制体体积	—、—、 m^3
t^*	全场最大含水率首次降至 0.15kg/kg 的时刻	s

四、 模型假设

本文模型建立在以下假设之上，各条均给出依据或适用范围。

- (1) 药材简化为各向同性均质圆柱，长度 25cm 恒定，温度与含水率沿轴向均匀，只研究径向一维传递。依据是长径比达 12.5，端面影响另有二维对照检验。
- (2) 横截面内为均匀径向收缩，材料点径向速度为 $v_r = (r/R)(dR/dt)$ 。附件只给出外半径，均匀应变是引入参数最少的内部运动学假设。
- (3) 实测半径作为已知外部输入，温度与含水率场不反向预测收缩，以避免引入未给定的力学本构关系。

(4) 干物质不参与反应、不随水分流失，各材料单元的干物质量守恒，这是水分脱除情境的基本质量约束。

(5) 初始质量由题给经验密度确定；此后湿体积密度由单元水量与实际体积计算。题给 $\rho(C)$ 不再兼作真实湿体积密度，该取舍在第 5.1 节说明。

(6) 水分迁移由单一有效扩散系数 $D(T,C)$ 描述，导热系数 $k(C)$ 与比热 $c_p(C)$ 取题给经验式，不引入题目未给出的液态水与蒸气分配参数。

(7) 环境沿药材侧面均匀，烘房温湿度为时间的已知函数；4h 以后缺少观测，按最后 1h 的均值延拓。

(8) 表面采用第三类边界条件，传质以折算到药材干基的环境平衡含水率为驱动值。附件未提供药材的吸附等温线，该解释属于有效闭合假设。

(9) 不计内部相变位置与结合水解吸热。作为机理对照，可假设净排出的水分全部在表面汽化，此时以 λ 表示汽化耦合系数。

(10) 忽略机械功、辐射换热以及设备散热与排风损失，单根药材的热量收支不等同于整机或整房能耗。

五、 模型建立与求解

5.1 统一建模准备

状态量与守恒约束。以药材中水分质量与干物质质量之比定义干基含水率 C 。烘房给出的水分浓度与药材含水率虽然都写作 kg/kg ，但分母分别为干空气质量与干药材质量，不能直接相减；本文将附件数值解释为已折算到药材干基的环境平衡含水率 C_{eq} 。题给密度 $\rho(C)$ 若同时充当真实湿体积密度，将与“指定几何、干基输运、干物质守恒”三者不相容，故以每个材料单元的干物质量守恒为硬约束：初始干物质量由题给密度确定，此后湿体积密度由单元水量与实际体积计算，相关检验见第六章。题给环境与物性数据见赛题及附件[1]。

设 ρ_d 为当前体积中的干物质质量密度， v 为干固体骨架速度， J_w 为相对骨架的水分质量通量，则干固体与水的连续方程分别为式（1）与式（2），水分通量由有效扩散表达式（3）给出。

$$\frac{\partial \rho_d}{\partial t} + \nabla \cdot (\rho_d v) = 0 \quad (1)$$

$$\frac{\partial (\rho_d C)}{\partial t} + \nabla \cdot (\rho_d C v + J_w) = 0 \quad (2)$$

$$J_w = -\rho_d D \nabla C \quad (3)$$

以式（1）消去式（2）中的骨架压缩项，得式（4）。式（4）表明干基含水率不会因几何压缩而自动升高：水分通量为零时水分随干固体同步运动， C 保持不变，因此不能额外添加与收缩率成正比的源项。

$$\rho_d \frac{DC}{Dt} = \nabla \cdot (\rho_d D \nabla C) \quad (4)$$

收缩几何与材料坐标。取材料坐标 $x = r/R(t)$ ，设圆柱长度 L 恒定、横截面内为均匀径向收缩，材料点速度 $v_r = (r/R)(dR/dt)$ 。此时体积比与干质量密度只随时间变化，如式（5）所示；湿体积密度由式（6）给出。

$$x = \frac{r}{R(t)}, \quad J = \left(\frac{R}{R_0}\right)^2, \quad \rho_d(t) = \frac{\rho_{d,0}}{J(t)} \quad (5)$$

$$\rho_{wet}(x, t) = \rho_d(t)[1 + C(x, t)] \quad (6)$$

在该均匀应变假设下 ρ_d 在空间上均匀，水分方程在材料坐标下化为式（7）。式中材料运动项与坐标变换项恰好抵消，不应重复加入收缩对流项。

$$\frac{\partial C}{\partial t} \Big|_x = \frac{1}{R^2 x} \frac{\partial}{\partial x} \left(x D \frac{\partial C}{\partial x} \right) \quad (7)$$

边界条件与初值。中心处由轴对称给出零通量条件，表面由对流传质给出水分通量，如式（8）所示；表面单位面积的水分质量通量按式（9）计算。该通量同时用于水分收支、显热输运与相变热的计算，以保证三个方程使用同一排水量。

$$\frac{\partial C}{\partial x} \Big|_{x=0} = 0, \quad -\frac{D}{R} \frac{\partial C}{\partial x} \Big|_{x=1} = h_m (C_s - C_{eq}) \quad (8)$$

$$j_w = \rho_d h_m (C_s - C_{eq}) \quad (9)$$

热边界与能量方程。以 0°C 为显热参考温度，控制体显热为 $m_d(c_s + c_w C)\theta$ 。忽略机械功与辐射后，径向能量守恒方程写为式（10）：导热沿温度梯度输运热量，水分流动同时携带显热，两项使用同一水分通量。

$$\frac{\partial[\rho_d(c_s + c_w C)\theta]}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial \theta}{\partial r} + c_w \theta \rho_d D \frac{\partial C}{\partial r} \right) \quad (10)$$

表面热边界由对流换热与汽化吸热共同决定，见式（11）；中心处为零通量，初值为 $\theta(x, 0) = 28^\circ\text{C}$ 、 $C(x, 0) = 2.55\text{kg/kg}$ 。式中 λ 为汽化耦合系数： $\lambda = 0$ 表示不计显式相变热， $\lambda = 1$ 表示净排出的水分全部在表面汽化。相变热只在表面扣除一次，不在体积内按同一失水量重复计入。

$$-\frac{k}{R} \frac{\partial \theta}{\partial x} \Big|_{x=1} = h(\theta_s - T_a) + \lambda L_v j_w \quad (11)$$

物性取值。题给比热可恒等改写为式（12），其中 c_s 与 c_w 分别为干物质与水的比热；四个问题所用经验公式的取用关系见表 2，其中 $T_K = T + 273.15$ 。

$$c_p(C) = \frac{c_s + c_w C}{1 + C} \quad (12)$$

表 2 药材物性经验公式的分问取用

问题	$\rho_{\text{emp}}/(\text{kg}\cdot\text{m}^{-3})$	$c_p/(\text{J}\cdot\text{kg}^{-1}\cdot\text{K}^{-1})$	$k/(\text{W}\cdot\text{m}^{-1}\cdot\text{K}^{-1})$	$D/(\text{m}^2\cdot\text{s}^{-1})$
—	820	2600	0.36	$7 \times 10^{-9} \exp(-0.89/C)$
二、三	$650+128C$	$1450+2736C/(1+C)$	$0.21+0.38C/(1+C)$	$2.4 \times 10^{-3} \exp(-0.45/C - 3850/T_K)$
四	$760+90C$	$1850+2150C/(1+C)$	$0.12+0.20C/(1+C)$	$4.2 \times 10^{-4} \exp(-0.30/C - 3850/T_K)$

5.2 问题一模型建立与求解

问题一取表 2 中问题一的物性：密度 $820\text{kg}\cdot\text{m}^{-3}$ 、比热 $2600\text{J}\cdot\text{kg}^{-1}\cdot\text{K}^{-1}$ 、导热系数 $0.36\text{W}\cdot\text{m}^{-1}\cdot\text{K}^{-1}$ ，扩散系数 $D = 7 \times 10^{-9} \exp(-0.89/C)$ 只随含水率变化。药材长度 25cm、半径恒为 2cm，材料坐标与径向坐标重合，初值取 28°C 与 2.55kg/kg ，控制方程与边界条件用式（7）~式（11）。

求解至 1800s，表 3 与表 4 给出题面要求时刻与位置上的温度与水分浓度，逐秒、径向每 0.1cm 的完整结果保存为 result1.xlsx。

温度与含水率的演化如图 3 所示。30min 内中心含水率始终等于初值，水分变化集中在表层，表面含水率在 1800s 时降至 1.5102kg/kg。含水率解几乎不受是否计入相变潜热的影响（两种情况的最大差为 2.079×10^{-7} ），但温度场差异显著：计入表面汽化热后表面温度在约 7min 内降到 10℃ 以下，1800s 时中心为 11.2204℃、表面为 12.7079℃；不计相变热时同时刻中心与表面分别升至 33.7170℃ 与 36.9727℃。

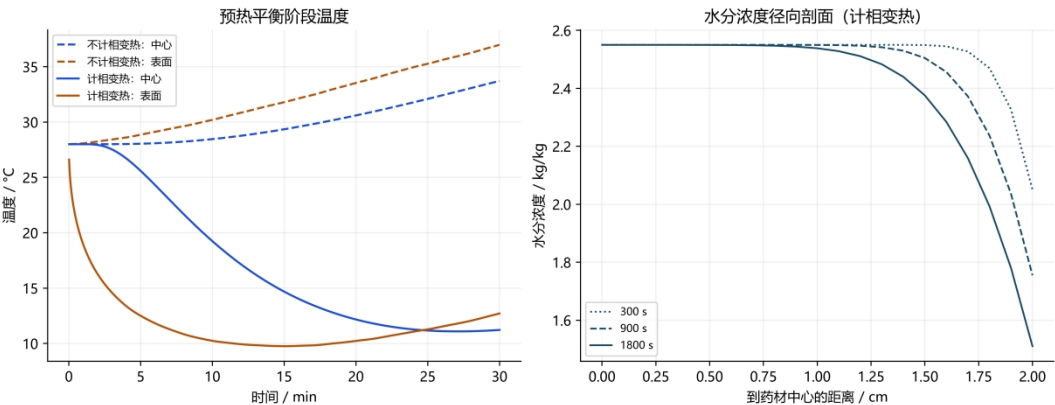


图 3 预热平衡阶段的温度与水分浓度

上述降温来自“分全部在表面汽化”与“环境水分浓度已折算”两项假设，其与露点的一致性在第六章讨论。因此表 3 的温度是既定假设下的机理结果，不宜直接当作药材内部的实际温度。

表 330 分钟内药材的温度（单位：℃）

时间/s	0	0.5	1	1.5	2
100	27.9780	27.8967	27.2290	24.2973	17.0079
300	25.5914	24.8273	22.4053	18.1740	12.5009
600	19.2385	18.5440	16.5628	13.6116	10.2380
900	14.6820	14.2653	13.1074	11.4711	9.7532
1200	12.1666	11.9652	11.4295	10.7551	10.2304
1500	11.1676	11.1231	11.0367	11.0370	11.2734
1800	11.2204	11.2767	11.4776	11.9169	12.7079

表 430 分钟内药材的水分浓度（单位：kg/kg）

时间/s	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2470

时间/s	0	0.5	1	1.5	2
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

5.3 问题二模型建立与求解

问题二统一采用表 2 中问题二、三的物性，扩散系数同时依赖温度与含水率，体现温度与水分双向耦合。控制方程与边界条件仍为式 (7)~式 (11)，但需从 $t = 0$ 用该组物性重新求解，不能以问题一的终点值作为初值。

表 5 与表 6 给出 3h 内每隔 0.5h 的径向温度与水分浓度，逐秒、径向每 0.1cm 的完整结果保存为 result2.xlsx。

表 53 小时内药材的温度（单位：℃）

时间/h	0	0.5	1	1.5	2
0.5	12.2261	12.1942	12.1377	12.1728	12.4712
1	16.8517	17.0169	17.5093	18.3203	19.4618
1.5	22.9376	23.0552	23.4024	23.9603	24.6912
2	26.8739	26.9471	27.1612	27.5016	27.9498
2.5	29.3482	29.3942	29.5272	29.7375	30.0501
3	31.1631	31.2003	31.3115	31.4927	31.7414

表 63 小时内药材的水分浓度（单位：kg/kg）

时间/h	0	0.5	1	1.5	2
0.5	2.5500	2.5500	2.5485	2.4548	1.3397
1	2.5499	2.5484	2.5143	2.2088	1.1679
1.5	2.5441	2.5261	2.4026	1.9671	1.0876
2	2.5029	2.4524	2.2457	1.7766	1.0178
2.5	2.4054	2.3313	2.0837	1.6240	0.9504
3	2.2680	2.1849	1.9290	1.4949	0.8882

3h 时中心温度 31.1631°C、表面温度 31.7414°C，表层含水率降至 0.8882kg/kg，中心仍为 2.2680kg/kg。与不计相变热的基准相比，同时刻中心温度由 49.8495°C 降至 31.1631°C：蒸发吸热使药材明显低于烘房温度，而这一温差又通过扩散系数 $D(T,C)$ 反过来抑制水分迁移，这正是相变耦合不可忽略的原因，前 3h 的温度对照见图 4。

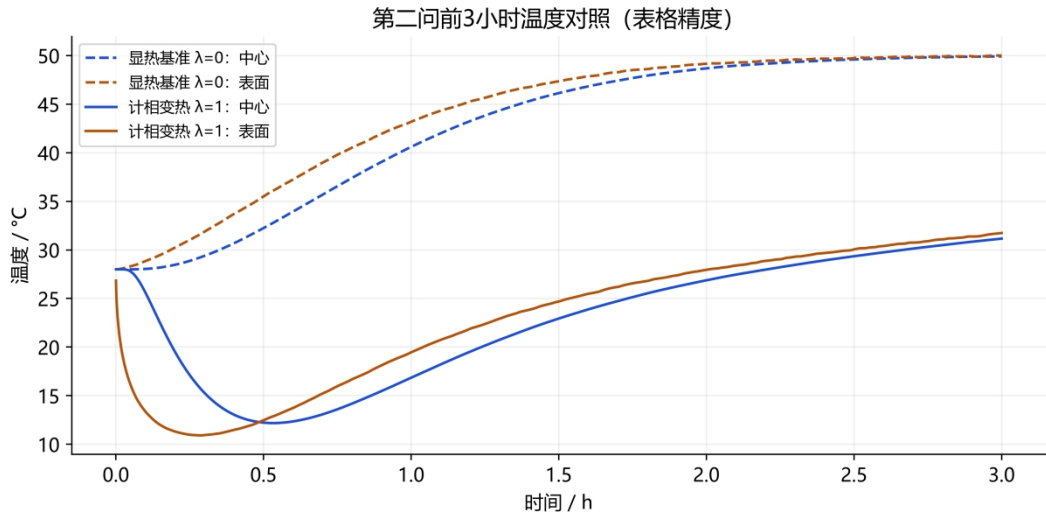


图 4 第二问前 3 小时中心与表面温度对照

5.4 问题三模型建立与求解

问题三以全场最大含水率首次降至 0.15kg/kg 作为烘干终点，即 $t^* = \inf\{t:\max_x C(x,t)\leq 0.15\}$ ，物性仍取表 2 中问题二、三一行，半径恒为 2cm。求解时对全体控制体逐点比较含水率。

计算得临界时刻 60.430002h，严格低于阈值的首个整分钟为 60h26min，该整分钟的全场最大含水率为 0.14999651。表 7 给出每隔 6h 与终止时刻的径向水分浓度，每隔 60s、径向每 0.1cm 的完整结果保存为 result3.xlsx。

表 7 药材烘干过程的水分浓度（问题三，单位：kg/kg）

时间/h	0	0.5	1	1.5	2
6	1.4099	1.3563	1.1988	0.9449	0.5836
12	0.5917	0.5731	0.5165	0.4165	0.2232
18	0.3470	0.3379	0.3093	0.2559	0.1013
24	0.2602	0.2544	0.2359	0.2003	0.0718

时间/h	0	0.5	1	1.5	2
30	0.2188	0.2144	0.2004	0.1729	0.0622
36	0.1944	0.1908	0.1793	0.1563	0.0579
42	0.1782	0.1751	0.1651	0.1450	0.0556
48	0.1665	0.1637	0.1548	0.1367	0.0542
54	0.1575	0.1550	0.1469	0.1304	0.0533
60	0.1505	0.1481	0.1406	0.1252	0.0527
烘干结束时间	0.1500	0.1477	0.1402	0.1249	0.0526

终点判据的选择对结果影响很大。若改用截面平均含水率判定，平均含水率在 38.912h 即达到 0.15，而此时中心含水率仍为 0.18581，比全域达标提前 21.518h 停机，得到的是中心尚未合格的药材。两种判据下该时刻的中心含水率几乎相同，差别完全来自判据本身，如图 5 所示。因此本文以全场最大含水率作为停机判据。

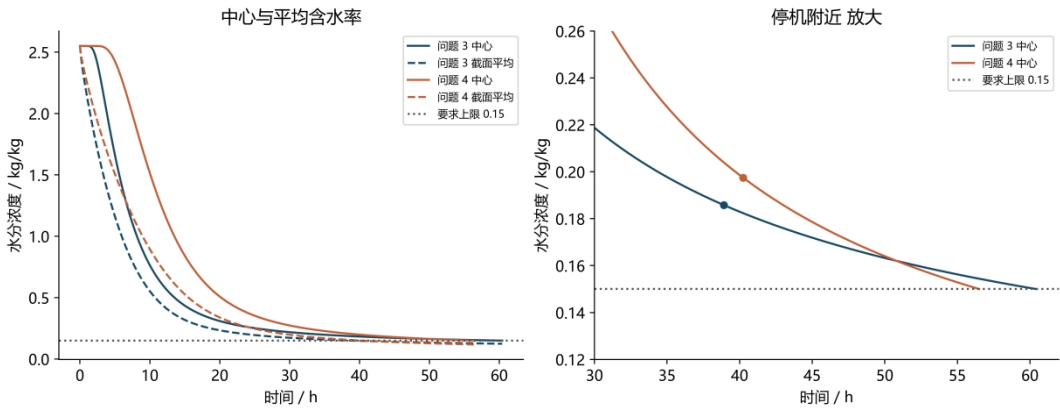


图 5 中心与平均含水率及停机附近放大

5.5 问题四模型建立与求解

问题四采用表 2 中问题四一行的物性，并按附件 2 的实测半径把表面位置作为已知输入。半径由 2cm 单调收缩至 1.198cm，对应体积比 $J = (R/R_0)^2 \approx 0.359$ ，即体积减少约 64%；求解在材料坐标下进行，物理位置超出当前半径时该位置不属于药材。

计算得临界时刻 56.501190h，严格低于阈值的首个整分钟为 56h31min，该整分钟的全场最大含水率为 0.14997156，表面含水率为 0.052591。表 8 给出每隔 6h 与终止时刻的结果，位于药材之外的位置留空；完整结果保存为 result4.xlsx。

表 8 药材烘干过程的水分浓度（问题四，单位：kg/kg）

时间/h	0	0.5	1	1.5	2	药材表面
6	2.2768	2.0493	1.3151	—	—	0.3819
12	1.1956	1.0365	0.5935	—	—	0.2178
18	0.6126	0.5418	0.3278	—	—	0.1224
24	0.3758	0.3395	0.2211	—	—	0.0806
30	0.2733	0.2506	0.1726	—	—	0.0651
36	0.2209	0.2045	0.1463	—	—	0.0587
42	0.1898	0.1770	0.1300	—	—	0.0556
48	0.1694	0.1587	0.1189	—	—	0.0539
54	0.1549	0.1457	0.1108	—	—	0.0529
烘干结束时间	0.1500	0.1412	0.1081	—	—	0.0526

第四问的烘干时长比第三问短 3.929h。这一差异同时来自物性改变与尺寸收缩两个原因：在固定问题四物性、仅取消收缩的对照中，时长由 51.0891h 增至 129.8502h，说明收缩本身显著加快干燥，而两问物性不同，因此不能把两问时长之差全部归因于收缩。问题三与问题四在水分布局上的差别见图 6。

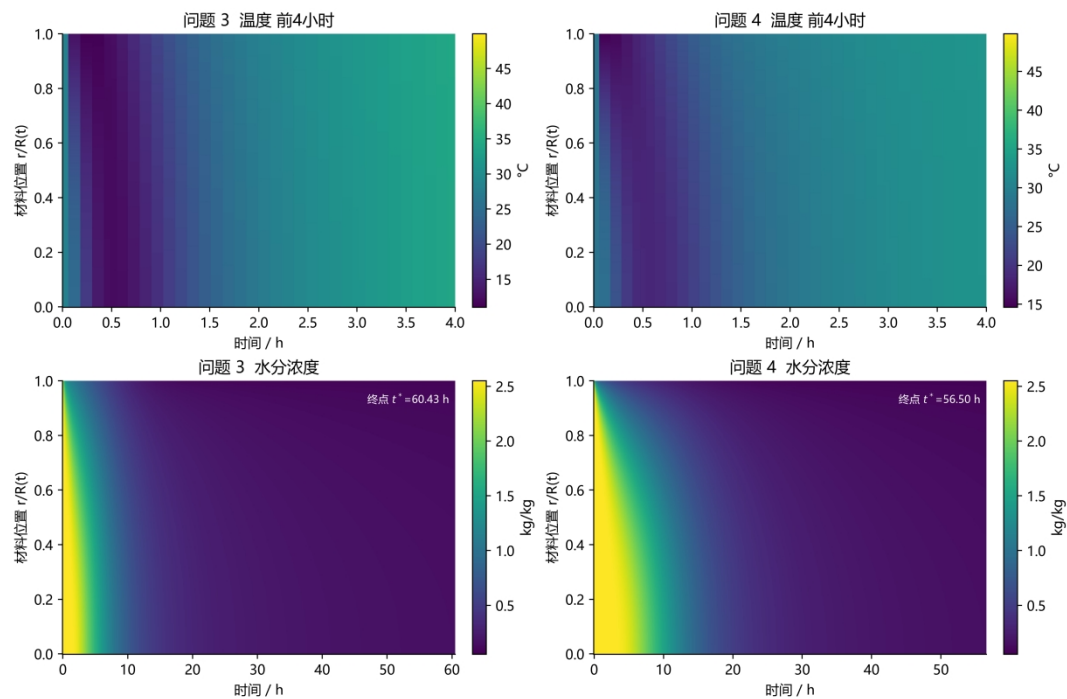


图 6 问题三与问题四的温度与水分浓度分布（材料坐标）

5.6 数值求解方法

空间离散。在材料坐标上采用环形有限体积离散，控制体的权重与体积由式 (13) 确定。相邻控制体共享同一面通量并以相反符号计入，逐单元求和后内部通量严格抵消，总水量的变化只由外边界排水决定。

$$w_i = \frac{x^{i+1/2} - x^{i-1/2}}{2}, \quad V_i(t) = 2\pi LR^2(t)w_i \quad (13)$$

水分方程离散。以 F 表示面上的水质量流率，单元水分收支、面通量与边界条件分别为式 (14) — 式 (16)。中心面面积为零，直接施加零通量条件，从而避免在 $r = 0$ 处计算 $1/r$ 。

$$m_{d,i} \frac{dC_i}{dt} = F_{i-1/2} - F_{i+1/2} \quad (14)$$

$$F_{i+1/2} = x_{i+1/2} D_{i+1/2} \frac{C_{i+1} - C_i}{\Delta x} \quad (15)$$

$$F_{-1/2} = 0, \quad F_{N+1/2} = -Rh_m(C_N - C_{eq}) \quad (16)$$

面扩散系数沿相邻节点的状态连线用三点 Gauss 积分计算，见式 (17)。恒温时它等价于扩散势 $\Phi(C) = \int D dC$ 的差商，可较稳妥地处理干表层低扩散系数造成的阻力；导热系数在相邻节点之间取调和平均值。

$$D_{i+1/2} = \int_0^1 D(T_i + s\Delta T, C_i + s\Delta C) ds \quad (17)$$

表面加密网格。干表层的扩散系数随含水率迅速衰减，故在表面附近加密网格，取式 (18) 所示的非均匀节点分布，其中 u 在 $[0,1]$ 上均匀取值。

$$x(u) = 1 - \frac{\sinh[4(1-u)]}{\sinh 4} \quad (18)$$

能量方程离散。能量方程按式 (19) 离散：面导热流率由相邻节点的温度差与调和平均导热系数给出，内部面的水分显热流量取相邻温度的平均值，表面对外排水取表面温度。由式 (19) 反解温度导数时扣除水分带走的显热项，避免失水过程凭空产生或消失显热。

$$\frac{dH_i}{dt} = (Q + E_w)_{i-1/2} - (Q + E_w)_{i+1/2} \quad (19)$$

时间积分与事件定位。空间离散后得到刚性常微分方程组,采用稀疏隐式 BDF 方法积分,相对容差 2×10^{-8} 、绝对容差 2×10^{-10} ,前 4h 最大内部步长 30s,其后为 600s。烘干终点以事件定位求解,事件函数为全场最大含水率与阈值之差;另用 Radau 方法在同一网格上交叉核验时间积分的一致性。

输出与舍入。输出时刻由连续解的插值给出,与积分器内部步长无关;结果按题面要求保留四位小数,问题四中位于当前半径之外的位置留空。所采用的网格、容差与收敛性证据见第六章。

六、 模型检验与灵敏度分析

6.1 模型检验与误差分析

为确认程序求解的是所写的方程,本文做了三项彼此独立的检验。解析基准:在常系数、均匀初值、恒定环境的简化问题下,用圆柱 Robin 边界条件的 Bessel 级数解作参照(特征根满足 $\mu J_1(\mu) = BiJ_0(\mu)$,取前 40 个根), $N=1280$ 时温度场的最大无量纲误差为 1.155×10^{-7} ,等效温度误差 $2.54 \times 10^{-6}^{\circ}\text{C}$ 。网格收敛:主工况在 160、320、640、1280 五档网格上重算,结果见表 9。时间积分对照:同一网格改用 Radau 方法重算,与 BDF 的临界时长差为 0.000189s 与-0.001352s,说明时长不是某一积分器的偶然结果。

表 9 网格加密与收敛检验

问题	网格 N	临界时长/h	较上一档时长差/s	最大温度差/ $^{\circ}\text{C}$	最大含水率差
第三问	160	60.429870	—	—	—
第三问	320	60.429970	0.363213	1.285e-03	1.386e-04
第三问	640	60.429995	0.090078	3.920e-04	4.405e-05
第三问	1280	60.430002	0.022267	9.240e-05	1.028e-05
第四问	160	56.500974	—	—	—
第四问	320	56.501136	0.580115	9.385e-04	1.157e-03
第四问	640	56.501178	0.151491	2.699e-04	2.798e-04
第四问	1280	56.501190	0.044446	6.381e-05	6.941e-05

最后一次加密使第三、四问的临界时长仅变化 0.022267s 与 0.044446s，全部导出时空点上的温度差不超过 $9.2 \times 10^{-5}^{\circ}\text{C}$ 、含水率差不超过 1.2×10^{-3} 。需要说明的是，相邻网格之差不是解析误差上界，四位小数也不因此等同于真解的舍入值。

6.2 守恒性与收支检验

模型以每个材料单元的干物质量恒定作为结构约束。逐时间点、逐控制体用湿体积密度除以 $1+C$ 再乘当前体积重建干质量，相对偏差为 0。水与能量另用独立的边界通量时间积分核对：通量以 10s 与 5s 两种步长采样，并在 4h 的环境分段处分开积分，再与初末库存差比较，结果见表 10。水量残差主要来自独立积分的截断误差，不能通过强制把残差写零替代检查。

表 10 单根药材的能量收支

题问	对流供热/kJ	汽化热/kJ	排水显热/kJ	储存显热变化/kJ	收支残差/J
第三问	506.9897	498.8413	28.9804	-20.8306	1.3996
第四问	517.0807	506.7652	29.7210	-19.4031	2.3723

6.3 极限工况与结构检验

三项极限工况用于检查方程结构而非数值精度。密闭、绝热且仅收缩时，水分与热量都不跨越边界，含水率与温度保持初值不变，而干质量密度随体积缩小而增大，这直接检验了干基方程并没有被加入与收缩率成正比的压缩源项。有失水但绝热、且不计显式相变热时，水分带走的显热与剩余材料的储能变化相抵，材料温度保持 28°C 不变，检验了显热通量与储能链式求导没有重复或遗漏。全场含水率始终非负、不超过初值，径向分布不出现反常升高。

端面影响用 40×40 的二维有限圆柱做对照：第三问一维与二维的临界时长相差 -7.75s ，第四问相差 -0.045s ，且两种情形的最大含水率都位于几何中心。这说明一维径向近似可用于判断最迟达标时间，但不表示端面附近的温湿场都能由一维模型代表，该对照也未做二维自身的网格无关性验证。

6.4 误差来源与物理闭合诊断

误差可分为三类：1.模型与假设误差，来自环境边界、相变位置与内部应变等假设；2.数据误差，来自附件只提供烘房温湿度与半径；3.计算误差，来自空间离散与时间积分。

本问题中前两类远大于第三类：最后一次网格加密只改变 0.02—0.04s，而是否计入相变热改变 2.96h 与 5.39h，边界情景更可改变数十小时。因此提高结果可信度的重点是交代闭合条件，而不是继续加密网格。

还存在一个必须公开的物理一致性问题。若把附件的水分浓度按真实空气含湿量解释、总压取 101325Pa，则后期 49.99893°C 与 0.04998754 对应相对湿度约 61.03%、露点约 40.38°C；而主工况的解在第三问 0.002—7.067h、第四问 0.001—11.784h 出现表面温度低于该露点却仍在向外排水的时段（采样最低表温分别为 10.913°C 与 14.551°C，最大 $p_v/p_{\text{sat}}(T_s)$ 分别为 3.543 与 2.396）。

该诊断附带两个条件：题面本意是否为己折算的有效浓度、界面是否为常规液态或吸附水。这说明有效浓度边界不能同时充当真实界面的驱动力；工程化的做法是改用 $j_w = \beta_p[a_w(C_s, T_s)p_{\text{sat}}(T_s) - p_v]$ ，但本题未给出水活度关系与气膜系数，无法据此标定。

此外，附件没有药材内部的温度、含水率或称重观测，本文只能证明数值求解与收支一致，不能声称预测精度已被实验验证。

6.5 灵敏度分析

参数扰动在经验闭合基准、N=160 同网格下进行，以免把网格误差混入参数影响，结果见表 11，各情景的时长变化见图 7。

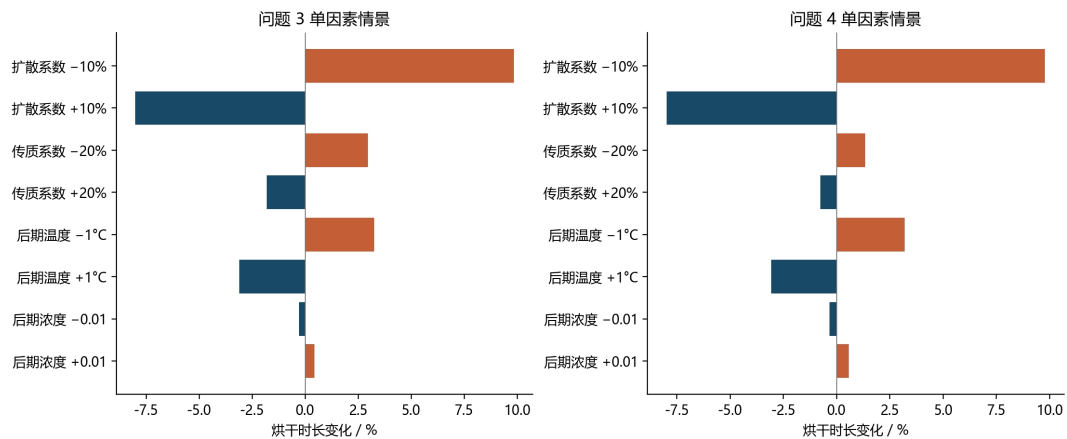


图 7 单因素情景下的烘干时长变化（经验闭合基准）

表 11 单因素情景的灵敏度（经验闭合基准，N=160）

情景	第三问时长/h	第三问变化/%	第四问时长/h	第四问变化/%
扩散系数-10%	63.1330	+9.84	56.0890	+9.79
扩散系数+10%	52.8672	-8.02	47.0094	-7.99
传质系数-20%	59.1711	+2.95	51.7749	+1.34
传质系数+20%	56.4355	-1.81	50.6968	-0.77
后期温度-1°C	59.3420	+3.25	52.7264	+3.20
后期温度+1°C	55.6890	-3.11	49.5225	-3.07
后期浓度-0.01	57.3056	-0.29	50.9149	-0.34
后期浓度+0.01	57.7209	+0.43	51.3781	+0.57

可见扩散系数最敏感： $\pm 10\%$ 的扰动使时长变化约 -8.0% 与 $+9.8\%$ ；后期温度次之， $\pm 1^\circ\text{C}$ 约带来 $\pm 3.2\%$ 的变化；传质系数与后期浓度的影响较小，分别不超过 3.0% 与 0.6% 。在固定问题四物性而取消收缩的对照中，时长由 51.0891h 增至 129.8502h ，说明收缩本身显著加快干燥；把表面汽化耦合系数从 0 增至 1 ，第三、四问的时长分别单调延长 2.96h 与 5.39h 。

表 12 有效环境边界的压力情景

情景	第三问/h	第四问/h
后期有效平衡含水率改为 0.10	61.1785	55.0519
后期有效平衡含水率改为 0.14	85.5356	75.6260
四因素联合不利情景	67.1564	58.9147

表 12 中的情景是人为设定的压力试验，不是经概率标定的分布，因此本文不给出置信区间或“最坏情况”结论。还有一个只依赖判据本身的条件性结论：若最终稳定环境对应的平衡含水率不低于 0.15，而初始各处含水率都高于 0.15，则仅靠当前的扩散脱水机制不能在有限时间内保证各处严格低于 0.15——平衡边界的物理含义影响的是可达性，而不只是烘干时长的长短。

七、 模型改进与推广

7.1 模型改进方向

一是把表面传质改为 $j_w = \beta_p[a_w(C_s, T_s)p_{\text{sat}}(T_s) - p_v]$ ，并通过解吸等温线把空气相对湿度映射到药材平衡含水率，使质量与潜热使用同一方向正确的通量。二是补充内部温湿度或称重观测用于标定有效扩散系数与界面系数。三是把完整汽化模型扩展到二维有限圆柱并做网格无关性验证。四是以区间或概率方式处理多参数不确定性，替代当前的单因素情景。

7.2 模型推广

该框架适用于形状规则、可近似为一维径向传递的农产品热风干燥，可直接用于比较不同烘房温湿度历程下的烘干时长，或筛选停机判据与工艺参数。推广的前提是先取得该材料的有效扩散系数、解吸关系与界面传质系数；缺少这些关系时，模型给出的是既定假设下的规律，而不是可直接投产的工艺参数。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码设计、论文思路提示和提交目录梳理，详细使用情况见支撑材料。

八、 参考文献

全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛题目 A 题: 药材烘干问题[Z]. 2026.

Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.

International Association for the Properties of Water and Steam. Revised Release on the IAPWS Formulation 1995 for the Thermodynamic Properties of Ordinary Water Substance for General and Scientific Use[R]. 2018.

九、 附录

A 支撑材料目录

(1) 原始题目与数据：A 题.pdf、附件 1.xlsx（烘房环境历程）、附件 2.xlsx（药材半径历程）。

(2) 结果附件：result1.xlsx、result2.xlsx、result3.xlsx、result4.xlsx。四份文件均对应正文采用的计表面汽化热主工况（ $\lambda=1$ ）。

(3) 计算核验材料：主工况结果 JSON、逐格回读核验记录 checks.json 及输入清单。

B 代码

文件名：export_latent_books.py

```
"""λ=1 主工况（计全部表面汽化热）的四问交付导出与核验。

背景：论文主结果取"干质量守恒热湿模型 + 表面全部汽化热"（λ=1，L_v=2.38 MJ/kg），
第三、四问临界时长 60.430002 h / 56.501190 h。原 result1-4.xlsx 属 λ=0 显热基准，
不能与主工况数字混用，故本脚本按同一求解器重新求解四问并导出对应交付文件。

产物

results/result1.json - result4.json    与 守恒热湿模型 同格式的结果表数据
results/tables_1_6.json                论文表 1-6 的数值
results/checks.json                    求解、守恒、逐格回读核验记录
result1.xlsx - result4.xlsx            交付工作簿（表头/列样式克隆自显热基准交付文件）

用法

python export_latent_books.py --cache <临时缓存目录>

可选参数

--n 1280                求解网格：默认 1280，与 60.430002 h / 56.501190 h 同网格
--style-source DIR      克隆表头与列样式的来源目录，默认 计算与证据/守恒热湿模型
--skip-xlsx             只求解与核验，不写工作簿
"""

import argparse
import json
```

```

import sys

import time

import zipfile

from pathlib import Path

from xml.sax.saxutils import escape

import numpy as np

HERE = Path(__file__).resolve()

OUT_ROOT = HERE.parents[1]          # 计算与证据/相变热工况交付
EVIDENCE = HERE.parents[2]         # 计算与证据
CONSERVATIVE_CODE = EVIDENCE / '守恒热湿模型' / 'code'
sys.path.insert(0, str(CONSERVATIVE_CODE))

from conservative_model import ConservativeSettings as Settings # noqa: E402
from run_conservation_study import get_model, payload, extend, balance_checks # noqa: E402

RESULTS = OUT_ROOT / 'results'

# 文档中记录的  $\lambda=1$ 、 $N=1280$  临界时长，用于核对本次复算是否一致。
DOCUMENTED = {3: 60.430002, 4: 56.501190}
CRITICAL_TOLERANCE_H = 0.01
R_COLS_CM = np.round(np.arange(0, 2.0001, .1), 1)
TABLE_R_CM = [0., .5, 1., 1.5, 2.]

def save_lf(path, value):
    """写 JSON，换行统一为 LF，与成果中其余文本文件一致。"""
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False),
                    encoding='utf-8', newline='\n')

# ----- 工作簿写入

def _col_letter(index):
    """0 -> A, 25 -> Z, 26 -> AA"""
    letters = ''
    index += 1
    while index:
        index, rem = divmod(index - 1, 26)
        letters = chr(65 + rem) + letters
    return letters

```

```

def _num(value):
    """数值单元格文本：避免科学计数法，保持最短往返表示。"""
    if isinstance(value, int):
        return str(value)
    text = json.dumps(float(value))
    if 'e' in text or 'E' in text:
        text = ('%.10f' % float(value)).rstrip('0').rstrip('.')
    return text

def build_rows_xml(rows, row_styles, header_xml):
    """由数据行生成 sheetData 内容：row_styles 为每列样式索引的列表。"""
    parts = [header_xml]
    for offset, row in enumerate(rows[1:], start=2):
        cells = []
        for col, value in enumerate(row):
            ref = '%s%d' % (_col_letter(col), offset)
            style = row_styles[col]
            if value is None:
                cells.append('<x:c r="%s" s="%s" />' % (ref, style))
            elif isinstance(value, str):
                cells.append('<x:c r="%s" s="%s" t="str"><x:v>%s</x:v></x:c>'
                             % (ref, style, escape(value)))
            else:
                cells.append('<x:c r="%s" s="%s" t="n"><x:v>%s</x:v></x:c>'
                             % (ref, style, _num(value)))
        parts.append('<x:row r="%d" ht="18" customHeight="1">%s</x:row>' % (offset, ''.join(cells)))
    return ''.join(parts)

def _sheet_parts(archive):
    """工作簿内各工作表名 -> 部件路径，按工作簿顺序。"""
    ns = {'m': 'http://schemas.openxmlformats.org/spreadsheetml/2006/main',
          'r': 'http://schemas.openxmlformats.org/officeDocument/2006/relationships',
          'p': 'http://schemas.openxmlformats.org/package/2006/relationships'}
    import xml.etree.ElementTree as ET
    wb = ET.fromstring(archive.read('xl/workbook.xml'))
    rels = ET.fromstring(archive.read('xl/_rels/workbook.xml.rels'))
    targets = {rel.get('Id'): rel.get('Target') for rel in rels.findall('p:Relationship', ns)}
    order = []
    for sheet in wb.find('m:sheets', ns):
        target = targets[sheet.get('{%s}id' % ns['r'])].lstrip('/')
        order.append((sheet.get('name'), target if target.startswith('xl/') else 'xl/' + target))
    return order

```

```

def clone_workbook(dst, style_source, sheets):

    """把 sheets{表名: 行列表} 写入 dst: 表头与列样式取自 style_source 的同名工作表。"""

    src = zipfile.ZipFile(style_source)

    mapping = dict(_sheet_parts(src))

    missing = set(sheets) - set(mapping)

    if missing:

        raise KeyError('样式来源缺少工作表: %s' % sorted(missing))

    replaced = {}

    for name, rows in sheets.items():

        part = mapping[name]

        xml = src.read(part).decode('utf-8')

        start = xml.index('<x:sheetData>') + len('<x:sheetData>')

        end = xml.index('</x:sheetData>')

        head = xml[:start]

        tail = xml[end:]

        # 表头行与第二行样式模板按原样取用, 保持与显热基准交付件一致的外观。

        row1_start = xml.index('<x:row r="1"')

        row1_end = xml.index('</x:row>', row1_start) + len('</x:row>')

        header_xml = xml[row1_start:row1_end]

        row2_start = xml.index('<x:row r="2"')

        row2_end = xml.index('</x:row>', row2_start) + len('</x:row>')

        row_styles = []

        import re

        for match in re.finditer(r'<x:c r="([A-Z]+)2"?: s="(\d+)"?', xml[row2_start:row2_end]):

            row_styles.append(match.group(2) or '0')

        if len(row_styles) < len(rows[0]):

            raise ValueError('%s 样式列数不足: %d < %d' % (name, len(row_styles), len(rows[0])))

        replaced[part] = (head + build_rows_xml(rows, row_styles, header_xml) + tail).encode('utf-8')

    with zipfile.ZipFile(dst, 'w', zipfile.ZIP_DEFLATED) as out:

        for item in src.infolist():

            data = replaced.get(item.filename, src.read(item.filename))

            out.writestr(item, data)

    src.close()

def read_book(path):

    """独立回读: 工作表名 -> 取值后的行列表 (空单元格为 None)。"""

    import xml.etree.ElementTree as ET

    ns = {'m': 'http://schemas.openxmlformats.org/spreadsheetml/2006/main'}

    archive = zipfile.ZipFile(path)

    shared = []

    if 'xl/sharedStrings.xml' in archive.namelist():

```

```

root = ET.fromstring(archive.read('xl/sharedStrings.xml'))

for si in root.findall('m:si', ns):

    shared.append(''.join(node.text or '' for node in si.iter(
        '{http://schemas.openxmlformats.org/spreadsheetml/2006/main}t')))

out = {}

for name, part in _sheet_parts(archive):

    root = ET.fromstring(archive.read(part))

    rows = []

    for row in root.iter('{%s}row' % ns['m']):

        values = []

        for cell in row.findall('m:c', ns):

            kind = cell.get('t')

            node = cell.find('m:v', ns)

            text = None if node is None else node.text

            if text is None:

                values.append(None)

            elif kind == 's':

                values.append(shared[int(text)])

            elif kind in ('str', 'inlineStr'):

                values.append(text)

            else:

                values.append(float(text))

        rows.append(values)

    out[name] = rows

archive.close()

return out

# ----- 表格与核验

def _pick(model, t, r_cm, field):

    value = float(model.profile(t, np.array([r_cm]), field)[0])

    return None if np.isnan(value) else round(value, 4)

def tables(models, criticals):

    m1, m3, m4 = models[1], models[3], models[4]

    tables_out = {}

    q1_times = [100, 300, 600, 900, 1200, 1500, 1800]

    q1 = {'时间/s': q1_times, '到药材中心的距离/cm': TABLE_R_CM}

    q1['温度/°C'] = [[_pick(m1, t, r, 'T') for r in TABLE_R_CM] for t in q1_times]

    q1['水分浓度/(kg/kg)'] = [[_pick(m1, t, r, 'C') for r in TABLE_R_CM] for t in q1_times]

    tables_out['表 1_表 2_问题 1'] = q1

```



```

q2_times = [1800, 3600, 5400, 7200, 9000, 10800]

q2 = {'时间/s': q2_times, '时间/h': [t / 3600 for t in q2_times], '到药材中心的距离/cm': TABLE_R_CM}

q2['温度/°C'] = [[_pick(m3, t, r, 'T') for r in TABLE_R_CM] for t in q2_times]

q2['水分浓度/(kg/kg)'] = [[_pick(m3, t, r, 'C') for r in TABLE_R_CM] for t in q2_times]

tables_out['表 3_表 4_问题 2'] = q2


for key, model in (('表 5_问题 3', m3), ('表 6_问题 4', m4)):
    q = model.p.problem

    critical = criticals[q]['critical_s']

    end_minute = criticals[q]['first_full_minute_s']

    grid = [t for t in np.arange(21600., critical + 1e-9, 21600.)]

    times = list(grid) + [end_minute]

    entry = {'时间/h': [round(t / 3600, 6) for t in times],
            '时间/s': [int(t) for t in times],
            '到药材中心的距离/cm': TABLE_R_CM,
            '行标签': ['%g' % (t / 3600) for t in grid] + ['烘干结束时间'],
            '水分浓度/(kg/kg)': [[_pick(model, t, r, 'C') for r in TABLE_R_CM] for t in times]}

    if q == 4:
        entry['药材表面'] = [round(float(model.state(t)[-1]), 4) for t in times]
        entry['当前半径/cm'] = [round(float(model.env.radius(t)) * 100, 6) for t in times]

    tables_out[key] = entry

return tables_out


def verify_book(path, expected, label):
    """逐格回读核验：表头、行列数、每个数值与空白位置。"""

    book = read_book(path)

    report = {'file': path.name, 'sheets': {}, 'status': 'PASS'}

    for name, rows in expected.items():
        got = book[name]

        if len(got) != len(rows):
            raise AssertionError('%s/%s 行数 %d != %d' % (label, name, len(got), len(rows)))

        numeric = 0
        blank = 0

        for i, (got_row, exp_row) in enumerate(zip(got, rows)):
            if len(got_row) < len(exp_row):
                raise AssertionError('%s/%s 第%d 行列数 %d < %d' % (label, name, i + 1, len(got_row), len(exp_row)))

            if i == 0:
                if [str(v) for v in got_row[:len(exp_row)]] != [str(v) for v in exp_row]:
                    raise AssertionError('%s/%s 表头不一致: %s' % (label, name, got_row[:6]))

                continue

            for j, exp in enumerate(exp_row):
                value = got_row[j]

```

```

        if exp is None:

            if value is not None:

                raise AssertionError('%s/%s 第%d 行第%d 列应为空白, 实为 %r'

                                    % (label, name, i + 1, j + 1, value))

            blank += 1

        else:

            if value is None or abs(float(value) - float(exp)) > 1e-12:

                raise AssertionError('%s/%s 第%d 行第%d 列 %r != %r'

                                    % (label, name, i + 1, j + 1, value, exp))

            numeric += 1

    report['sheets'][name] = {'rows': len(got), 'numeric_cells': numeric, 'blank_cells': blank}
    report['total_cells'] = sum(v['rows'] * len(expected[k][0]) for k, v in report['sheets'].items())
    report['numeric_cells'] = sum(v['numeric_cells'] for v in report['sheets'].values())
    report['blank_cells'] = sum(v['blank_cells'] for v in report['sheets'].values())

    return report


def legacy_reader_check(style_source):

    """用同一读取器核对显热基准交付件, 确认读取器与文档记录的单元格总数一致。"""

    names = ['result1.xlsx', 'result2.xlsx', 'result3.xlsx', 'result4.xlsx']

    total = 0

    detail = {}

    for name in names:

        book = read_book(style_source / name)

        cells = 0

        for sheet, rows in book.items():

            width = max(len(r) for r in rows)

            cells += len(rows) * width

        last_times = []

        for sheet, rows in book.items():

            last_times.append(int(rows[-1][0]))

        detail[name] = {'cells': cells, 'last_time_s': last_times}

        total += cells

    return {'total_cells': total, 'detail': detail,

            'documented_total_cells': 704020, 'matches_documented': total == 704020}


# ----- 主流程


def main():

    parser = argparse.ArgumentParser()

    parser.add_argument('--cache', required=True)

    parser.add_argument('--n', type=int, default=1280)

```

```

parser.add_argument('--style-source', default=str(EVIDENCE / '守恒热湿模型'))
parser.add_argument('--skip-xlsx', action='store_true')

args = parser.parse_args()

cache = Path(args.cache)

cache.mkdir(parents=True, exist_ok=True)

style_source = Path(args.style_source)

RESULTS.mkdir(parents=True, exist_ok=True)

wall = {}

checks = {'mesh': args.n, 'latent_fraction': 1., 'latent_heat_J_kg': 2.38e6}

models = {}

for q in (1, 3, 4):

    tic = time.perf_counter()

    models[q] = get_model(Settings(problem=q, n=args.n, latent_fraction=1.), cache)

    wall['solve_q%d' % q] = round(time.perf_counter() - tic, 1)

criticals = {}

for q in (3, 4):

    model = models[q]

    critical = float(model.end)

    minute = float((np.floor(critical / 60) + 1) * 60)

    max_before = float(np.max(model.state(critical)[model.m:]))

    models[q].parts_backup = len(model.parts)

    extend(model, minute)

    criticals[q] = {

        'critical_s': critical,

        'critical_h': critical / 3600,

        'documented_h': DOCUMENTED[q],

        'difference_vs_documented_s': critical - DOCUMENTED[q] * 3600,

        'first_full_minute_s': minute,

        'first_full_minute_h': minute / 3600,

        'max_C_at_critical': max_before,

        'max_C_at_first_full_minute': float(np.max(model.state(minute)[model.m:])),

        'surface_C_at_first_full_minute': float(model.state(minute)[-1]),

    }

    drift = abs(criticals[q]['difference_vs_documented_s'])

    print('CRITICAL q%d %.6f h (documented %.6f, drift %.3f s) minute %.0f s maxC %.8f'

          % (q, criticals[q]['critical_h'], DOCUMENTED[q], drift, minute,

             criticals[q]['max_C_at_first_full_minute']), flush=True)

    if drift > CRITICAL_TOLERANCE_H * 3600:

        raise AssertionError('第%d 问临界时长与文档记录相差 %.1f s, 超过 %.0f s 容差'

                              % (q, drift, CRITICAL_TOLERANCE_H * 3600))

    if not criticals[q]['max_C_at_first_full_minute'] < 0.15:

```

```

        raise AssertionError('第%d 问首个整分钟最大含水率未严格低于 0.15' % q)

books = {
    'result1': {'温度': payload(models[1], range(1, 1801), 'T'),
                '水分浓度': payload(models[1], range(1, 1801), 'C')},
    'result2': {'温度': payload(models[3], range(1, 10801), 'T'),
                '水分浓度': payload(models[3], range(1, 10801), 'C')},
    'result3': {'Sheet1': payload(models[3], np.arange(60, models[3].end + 1, 60))},
    'result4': {'Sheet1': payload(models[4], np.arange(60, models[4].end + 1, 60), shrink=True)},
}

for name, book in books.items():
    save_if(RESULTS / (name + '.json'), book)

tables_out = tables(models, criticals)
save_if(RESULTS / 'tables_1_6.json', {
    'condition': 'dry-mass-conserving model with all net drainage evaporated at the surface, '
                'lambda=1, Lv=2.38 MJ/kg, N=%d' % args.n,
    'critical_h': {str(q): criticals[q]['critical_h'] for q in (3, 4)},
    'first_full_minute_h': {str(q): criticals[q]['first_full_minute_h'] for q in (3, 4)},
    'tables': tables_out})

checks['critical'] = {str(q): criticals[q] for q in (3, 4)}
checks['balance'] = {}
for q in (1, 3, 4):
    checks['balance'][str(q)] = balance_checks(models[q])
    row = checks['balance'][str(q)]
    print('BALANCE q%d dry %.3e water %.3e energy %.3e' % (
        q, row['max_global_dry_mass_relative_error'],
        row['independent_time_integrals'][-1]['relative_water_residual'],
        row['independent_time_integrals'][-1]['relative_energy_residual']), flush=True)

checks['legacy_reader'] = legacy_reader_check(style_source)
print('LEGACY reader total cells %d (documented 704020, match %s)'
      % (checks['legacy_reader']['total_cells'], checks['legacy_reader']['matches_documented']), flush=True)

if not args.skip_xlsx:
    checks['books'] = {}
    for name, sheets in books.items():
        dst = OUT_ROOT / (name + '.xlsx')
        tic = time.perf_counter()
        clone_workbook(dst, style_source / (name + '.xlsx'), sheets)
        report = verify_book(dst, sheets, name)
        report['write_and_verify_s'] = round(time.perf_counter() - tic, 1)
        report['bytes'] = dst.stat().st_size

```

```

        checks['books'][name] = report

        print('BOOK %s rows %s cells %d numeric %d blank %d'

              % (name, {k: v['rows'] for k, v in report['sheets'].items()},
                  report['total_cells'], report['numeric_cells'], report['blank_cells']), flush=True)

    total_numeric = sum(v['numeric_cells'] for v in checks.get('books', {}).values())

    checks['wall_seconds'] = wall

    checks['total_numeric_cells'] = total_numeric

    checks['status'] = 'PASS'

    save_if(REULTS / 'checks.json', checks)

    print('FINISHED numeric_cells %d wall %s' % (total_numeric, wall), flush=True)

if __name__ == '__main__':
    main()

```

文件名: `make_latent_figures.py`

""" $\lambda=1$ 主工况的论文插图: 分辨率提高, 字号与图幅比例与原图一致。

与原图的对应关系

经验闭合对照/figures/02_热湿时空分布.png → figures/02_热湿时空分布_计相变热.png
 经验闭合对照/figures/03_达标与提前停机风险.png → figures/03_达标与提前停机风险_计相变热.png
 守恒热湿模型/figures/前3小时温度对照.png → figures/前3小时温度对照_计相变热.png

做法: `figsize`、字号、配色、子图布局与面板标题全部沿用原脚本,

只把 `savefig.dpi` 由 `200/150` 提高到 `--dpi` (默认 `400`),

因此"文字占图幅的比例"不变, 而像素分辨率提高一倍以上。

同时导出 $\lambda=1$ 的全精度场 `results/profiles_full_precision_latent.npz` 与

停机对照 `results/stopping_latent.json`, 供其他插图或复核复用。

用法

```

python make_latent_figures.py --cache <临时缓存目录> [--dpi 400]

"""

import argparse

import json

import os

import sys

from pathlib import Path

import numpy as np

HERE = Path(__file__).resolve()

OUT_ROOT = HERE.parents[1]

EVIDENCE = HERE.parents[2]

```

```

CONSERVATIVE = EVIDENCE / '守恒热湿模型'

sys.path.insert(0, str(CONSERVATIVE / 'code'))

os.environ.setdefault('MPLCONFIGDIR', str(Path(os.environ.get('TEMP', '/tmp')) / 'v5_mpl_cache'))

import matplotlib # noqa: E402
matplotlib.use('Agg')

import matplotlib.pyplot as plt # noqa: E402

from matplotlib import font_manager # noqa: E402

from scipy.optimize import brentq # noqa: E402

from conservative_model import ConservativeSettings as Settings # noqa: E402

from run_conservation_study import get_model # noqa: E402

RESULTS = OUT_ROOT / 'results'
FIGURES = OUT_ROOT / 'figures'
COLORS = ['#184A67', '#C45E37', '#407B68', '#92624A']

def pick_font():
    available = {f.name for f in font_manager.fontManager.ttflist}
    for name in ['Microsoft YaHei', 'SimHei', 'Noto Sans CJK SC']:
        if name in available:
            return name
    return 'DejaVu Sans'

def load_models(cache, n):
    return {q: get_model(Settings(problem=q, n=n, latent_fraction=1.), cache) for q in (1, 3, 4)}

def export_fields(models):
    """λ=1 全精度场，键名与守恒热湿模型导出一致。"""
    fields = {}
    for q, m in models.items():
        ts = np.unique(np.r_[np.linspace(0, m.end, 501), 1800., 10800. if q != 1 else 1800.])
        yy = m.state(ts)
        rd, rw = [], []
        for t, C in zip(ts, yy[m.m:].T):
            a, b = m.densities(t, C)
            rd.append(a)
            rw.append(b)
        fields[f'q{q}_time_s'] = ts
        fields[f'q{q}_x'] = m.x
        fields[f'q{q}_T_C'] = yy[:m.m]
        fields[f'q{q}_C_dry_basis'] = yy[m.m:]

```

```

        fields[f'q{q}_radius_m'] = np.array([m.env.radius(t) for t in ts])

        fields[f'q{q}_dry_density'] = np.array(rd).T

        fields[f'q{q}_wet_density'] = np.array(rw).T

        fields[f'q{q}_dry_cell_mass_kg'] = m.dry_mass

    np.savez_compressed(RESULTS / 'profiles_full_precision_latent.npz', **fields)

    return fields

def stopping(models):
    """ $\lambda=1$  的停机对照：平均含水率达标时刻与该时刻的中心含水率。"""

    out = {}

    for q in (3, 4):
        m = models[q]

        threshold = brentq(lambda t: float(m.mean_C(t)) - .15, 0., m.end)

        out[str(q)] = {
            'critical_h': m.end / 3600,
            'mean_threshold_h': threshold / 3600,
            'center_C_at_mean_threshold': float(m.state(threshold)[m.m]),
            'premature_stop_h': (m.end - threshold) / 3600,
        }

    (RESULTS / 'stopping_latent.json').write_text(
        json.dumps(out, ensure_ascii=False, indent=2, allow_nan=False), encoding='utf-8', newline='\n')

    return out

def figure_spacetime(fields, stop, dpi):
    """对应原 02_热湿时空分布.png: 2x2, 材料坐标, 时间-材料位置伪彩图。"""

    fig, axs = plt.subplots(2, 2, figsize=(11, 7.2), layout='constrained')

    for col, q in enumerate([3, 4]):
        ts = fields[f'q{q}_time_s'] / 3600

        x = fields[f'q{q}_x']

        C = fields[f'q{q}_C_dry_basis']

        T = fields[f'q{q}_T_C']

        for row, mat, title, unit in [(0, T, '温度', '°C'), (1, C, '水分浓度', 'kg/kg')]:
            im = axs[row, col].pcolormesh(ts, x, mat, shading='auto', cmap='viridis')

            axs[row, col].set(xlabel='时间 / h', ylabel='材料位置 r/R(t)', title=f'问题 {q} {title}')

            if row == 0:
                axs[row, col].set_xlim(0, 4)

                axs[row, col].set_title(f'问题 {q} 温度 前 4 小时')

            else:
                # 水分浓度的横轴终点即烘干终点, 故用角标文字标注, 不画贴边虚线。

                tc = stop[str(q)]['critical_h']

                axs[row, col].text(.985, .955, f'终点  $t^* = {tc:.2f}$  h', transform=axs[row, col].transAxes,
                                ha='right', va='top', color='w', fontsize=8)

```

```

fig.colorbar(im, ax=axes[row, col], label=unit, pad=.02)

fig.savefig(FIGURES / '02_热湿时空分布_计相变热.png')

plt.close(fig)

def figure_stopping(fields, stop, dpi):
    """对应原 03_达标与提前停机风险.png: 中心与截面平均、停机附近放大。"""
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2), layout='constrained')
    for q, color in zip([3, 4], COLORS):
        ts = fields[f'q{q}_time_s'] / 3600
        C = fields[f'q{q}_C_dry_basis']
        x = fields[f'q{q}_x']
        faces = (x[1:] + x[:-1]) / 2
        w = np.diff(np.r_[0, faces, 1]) ** 2 / 2
        axes[0].plot(ts, C[0], color=color, label=f'问题 {q} 中心')
        axes[0].plot(ts, 2 * w @ C, color=color, ls='--', label=f'问题 {q} 截面平均')
        mt = stop[str(q)][f'mean_threshold_h']
        mc = stop[str(q)][f'center_C_at_mean_threshold']
        axes[1].plot(ts, C[0], color=color, label=f'问题 {q} 中心')
        axes[1].scatter([mt], [mc], color=color, s=24, zorder=5)
    for ax in axes:
        ax.axhline(.15, color='#555', ls=':', label='要求上限 0.15')
        ax.set(xlabel='时间 / h', ylabel='水分浓度 / kg/kg')
        ax.legend(fontsize=8)
    axes[0].set_title('中心与平均含水率')
    axes[1].set(title='停机附近 放大', xlim=(30, 62), ylim=(.12, .26))
    fig.savefig(FIGURES / '03_达标与提前停机风险_计相变热.png')
    plt.close(fig)

def figure_first_three_hours(dpi):
    """对应原 前3小时温度对照.png: 第二问前3小时, 显热基准与计相变热对照。"""
    with (OUT_ROOT / 'results/result2.json').open(encoding='utf-8') as fh:
        latent = np.asarray(json.load(fh)[f'温度'][1:], float)
    with (CONSERVATIVE / 'results/result2.json').open(encoding='utf-8') as fh:
        sensible = np.asarray(json.load(fh)[f'温度'][1:], float)
    fig, ax = plt.subplots(figsize=(8.5, 4.2), layout='constrained')
    for a, ls, tag in [(sensible, '--', '显热基准 λ=0'), (latent, '-', '计相变热 λ=1')]:
        ax.plot(a[:, 0] / 3600, a[:, 1], ls=ls, color='#1D4ED8', label=tag + ': 中心')
        ax.plot(a[:, 0] / 3600, a[:, -1], ls=ls, color='#B45309', label=tag + ': 表面')
    ax.set(xlabel='时间 / h', ylabel='温度 / °C', title='第二问前3小时温度对照 (表格精度)')
    ax.legend(fontsize=9)
    ax.grid(alpha=.2)
    fig.savefig(FIGURES / '前3小时温度对照_计相变热.png', bbox_inches='tight')

```



```

plt.close(fig)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--cache', required=True)
    parser.add_argument('--n', type=int, default=1280)
    parser.add_argument('--dpi', type=int, default=400)
    args = parser.parse_args()

    FIGURES.mkdir(parents=True, exist_ok=True)
    RESULTS.mkdir(parents=True, exist_ok=True)
    # 字号与原脚本一致：经验闭合对照 10 pt，守恒热湿模型 11 pt。
    plt.rcParams.update({'font.family': pick_font(), 'axes.unicode_minus': False, 'font.size': 10,
                        'axes.spines.top': False, 'axes.spines.right': False,
                        'figure.dpi': 200, 'savefig.dpi': args.dpi})
    print('font=%s dpi=%d' % (plt.rcParams['font.family'], args.dpi), flush=True)

    models = load_models(Path(args.cache), args.n)
    fields = export_fields(models)
    stop = stopping(models)
    print('stopping:', json.dumps(stop, ensure_ascii=False), flush=True)

    figure_spacetime(fields, stop, args.dpi)
    plt.rcParams.update({'font.size': 11})
    figure_stopping(fields, stop, args.dpi)
    plt.rcParams.update({'figure.dpi': 150})
    figure_first_three_hours(args.dpi)

    for name in ['02_热湿时空分布_计相变热.png', '03_达标与提前停机风险_计相变热.png',
                '前3小时温度对照_计相变热.png']:
        path = FIGURES / name
        print('%-40s %8d bytes' % (name, path.stat().st_size), flush=True)
    print('FIGURES OK')

if __name__ == '__main__':
    main()

```

文件名：render_tables_md.py

"""把 results/tables_1_6.json 渲染为可直接粘进论文的 Markdown (tables_1_6.md)。

用法：python render_tables_md.py

"""

```

import json

```

```

from pathlib import Path

HERE = Path(__file__).resolve()
OUT_ROOT = HERE.parents[1]
RESULTS = OUT_ROOT / 'results'

def cell(value):
    return '-' if value is None else ('%g' % value if isinstance(value, (int, float)) else str(value))

def grid(header, rows, labels, columns):
    lines = ['| ' + ' | '.join([header] + [cell(c) for c in columns]) + ' | ',
             '| ' + ' | '.join(['---'] * (len(columns) + 1)) + ' | ']
    for label, row in zip(labels, rows):
        lines.append('| ' + ' | '.join([str(label)] + [cell(v) for v in row]) + ' | ')
    return '\n'.join(lines) + '\n'

def main():
    data = json.loads((RESULTS / 'tables_1_6.json').read_text(encoding='utf-8'))
    tables = data['tables']

    out = ['# 表 1-表 6（主工况  $\lambda=1$ ，计全部表面汽化热）', '',
           '- 工况：干质量守恒热湿模型 + 表面全部汽化热， $\lambda=1$ ， $L_v=2.38$  MJ/kg， $N=\%$ '],
          data.get('N', 1280),
          '- 第三问临界时长  $\%$  s，严格达标首个整分钟  $\%$  s',
          (round(data['critical_h']['3'], 6), round(data['first_full_minute_h']['3'], 6)),
          '- 第四问临界时长  $\%$  s，严格达标首个整分钟  $\%$  s',
          (round(data['critical_h']['4'], 6), round(data['first_full_minute_h']['4'], 6)),
          '- 空白「-」表示该固定位置已在药材之外（第四问半径收缩所致），不是零，也不是漏算。', '']

    q1 = tables['表 1_表 2_问题 1']
    out += ['# 表 1 30 分钟内药材的温度（单位： $^{\circ}\text{C}$ ）', '',
            grid('时间/s', q1['温度/ $^{\circ}\text{C}$ '], q1['时间/s'], q1['到药材中心的距离/cm'])]

    out += ['# 表 2 30 分钟内药材的水分浓度（干基，单位：kg/kg）', '',
            grid('时间/s', q1['水分浓度/(kg/kg)'], q1['时间/s'], q1['到药材中心的距离/cm'])]

    q2 = tables['表 3_表 4_问题 2']
    out += ['# 表 3 3 小时内药材的温度（单位： $^{\circ}\text{C}$ ）', '',
            grid('时间/h', q2['温度/ $^{\circ}\text{C}$ '], ['%g' % t for t in q2['时间/h']], q2['到药材中心的距离/cm'])]

    out += ['# 表 4 3 小时内药材的水分浓度（干基，单位：kg/kg）', '',
            grid('时间/h', q2['水分浓度/(kg/kg)'], ['%g' % t for t in q2['时间/h']], q2['到药材中心的距离/cm'])]

    q3 = tables['表 5_问题 3']

```

```

out += ['### 表 5 药材烘干过程的水分浓度（问题 3，单位：kg/kg）', '',
        grid('时间/h', q3['水分浓度/(kg/kg)'], q3['行标签'], q3['到药材中心的距离/cm'])]

q4 = tables['表 6_问题 4']
columns = list(q4['到药材中心的距离/cm']) + ['药材表面']
rows = [list(r) + [s] for r, s in zip(q4['水分浓度/(kg/kg)'], q4['药材表面'])]
out += ['### 表 6 药材烘干过程的水分浓度（问题 4，单位：kg/kg）', '',
        grid('时间/h', rows, q4['行标签'], columns), '',
        '问题 4 的当前半径（cm）随行给出，用于识别随时间改变的药材表面位置：', '',
        grid('时间/h', [[r] for r in q4['当前半径/cm']], q4['行标签'], ['当前半径/cm'])]

(REULTS / 'tables_1_6.md').write_text('\n'.join(out), encoding='utf-8', newline='\n')
print('written', RESULTS / 'tables_1_6.md')

if __name__ == '__main__':
    main()

```